Understanding services and utilities

AI Interview Coach - a guided explanation of the code you built.

The JD parser was straightforward until we added services and utilities. Those layers introduce several Python ideas at once: functions passed to functions, generics, exception routing, and result wrappers. This note separates those ideas and follows one JD through the application.

Start with three questions

Layer	Question it answers
Chain	How do I make one parsing attempt?
Utility	How do I repeat an operation within a limit?
Service	Which parser runs, when do we switch, and what do we return?

The core idea

The service owns the JD-specific decisions. The utility owns the repeated-attempt loop. The chain owns the prompt and model composition. Splitting these responsibilities makes each piece smaller, but you need to understand their connections before the design feels simple.

```
UI: candidate submits a JD
  |
  v
Service: choose primary / fallback / safe outcome
  |
  v
Utility: run this operation up to N times
  |
  v
Chain: prompt -> Groq -> validated result
```

Read pages 2-5 for the mechanics. Pages 6-8 provide worked traces, tests, and a revision guide.

Based on the local implementation after Milestone 2, 10 November 2026. No application code was changed to create this note.

Follow one request

1. The UI creates the service

```
service = JDParsingService(
    primary=build_jd_parser(),
    fallback=build_jd_fallback_parser(),
    primary_max_attempts=3,
    fallback_max_attempts=2,
)
outcome = service.parse(jd_text)
```

Building a chain configures it; **invoke()** is what makes a model request. Passing both chains into the service is called dependency injection: we supply the tools the service needs instead of forcing it to construct them internally.

2. The service checks the input

parse() rejects non-string input with ValueError. It trims whitespace. Blank text or text longer than 30,000 characters returns a correction result with zero attempts. No chain runs in those cases.

3. One operation calls one chain

```
output = chain.invoke({"jd_text": jd_text})
return JDParseResult.model_validate(output)
```

The private helper _invoke() passes the input dictionary to the chain and validates its returned value. The chains already parse into JDParseResult; this second check protects the service boundary, including when fake chains return dictionaries in tests.

4. The service chooses the next step

A validated primary result returns immediately. If all retryable primary attempts fail, the service tries the fallback. If both pipelines exhaust, it returns a marked technical correction outcome. Rate limits and other unhandled provider HTTP errors leave through JDParsingError.

5. The UI interprets the outcome

The UI checks exhausted first, then status. A valid extraction becomes a pending review; it is not yet confirmed session state. The candidate confirms edited requirements separately. Neither the service nor the utility sets the session to ready.

The utility: one small loop

File: **src/ai_interview_coach/utils/reliability.py**. This module knows nothing about JDs, Groq, Streamlit, or seniority. Its job is to call an operation within a limit.

```
for attempt in range(1, max_attempts + 1):
    try:
        value = operation()
    except retry_on:
        continue

    return AttemptResult(
        value=value, attempts=attempt, succeeded=True
    )

return AttemptResult(
    value=None, attempts=max_attempts, succeeded=False
)
```

Read the loop in plain English

range(1, max_attempts + 1) counts from 1 through the limit. With a limit of 3, it yields 1, 2, 3. These are total attempts: the first call plus at most two retries.

operation() performs one attempt. If it returns normally, the utility immediately returns a success result. It does not inspect the business meaning of the returned value.

except retry_on catches only the configured exception types. **continue** moves to the next iteration. If the last attempt fails this way, execution leaves the loop and returns succeeded=False.

An exception not listed in retry_on escapes the utility. That is deliberate: a programming error should not automatically become five paid calls or a misleading correction message.

What the utility does not do

It does not switch chains, sleep between calls, log errors, or save the last exception. It only records the returned value, call count, and whether an attempt returned normally. Adding those other capabilities would be separate work.

Example with no LLM

```
result = run_bounded_attempts(
    lambda: "hello",
    max_attempts=3,
    retry_on=(ValueError,),
)
# value="hello", attempts=1, succeeded=True
```

Decode the Python syntax

Callable[[], T] and the letter T

```
def run_bounded_attempts[T](
    operation: Callable[[], T],
    *,
    max_attempts: int,
    retry_on: tuple[type[Exception], ...],
) -> AttemptResult[T]:
```

T is a type placeholder. If the operation returns JDParseResult, the result container holds JDParseResult. If it returns a string, the container holds a string. The helper can be reused without depending on one application model. This type-parameter syntax requires Python 3.12 or newer.

Callable[[], T] means a callable that takes no arguments and returns T. The ***** makes the following arguments keyword-only, making call sites easier to read. The tuple annotation describes a tuple of exception classes; the ellipsis allows any tuple length.

Why we use lambda

```
lambda: self._invoke(self._primary, cleaned)
```

This creates a function to run later. It takes no arguments because it already refers to self and cleaned from its surrounding scope. The utility calls that function again for each attempt.

```
# Equivalent longer spelling:
def operation():
    return self._invoke(self._primary, cleaned)
```

Without lambda (or a named function), calling _invoke immediately would send the request before the retry helper gets control.

What the dataclass gives us

```
@dataclass(frozen=True)
class AttemptResult[T]:
    value: T | None
    attempts: int
    succeeded: bool
```

A dataclass generates initialization and a readable representation. **frozen=True** prevents reassignment of its fields; it does not deeply freeze objects inside value. **T | None** means a value of type T or no value. Unlike Pydantic models, dataclasses do not automatically validate these type hints at runtime.

The service: JD-specific decisions

File: **src/ai_interview_coach/services/jd_parser.py**. Read parse() as a decision tree rather than trying to memorize the whole class.

```
Input checks
  | acceptable text
  v
PRIMARY: up to 3 total attempts
  | success ----------------------> return result
  | retryable attempts exhausted
  v
FALLBACK: up to 2 total attempts
  | success ----------------------> return result
  | retryable attempts exhausted
  v
Return correction with exhausted=True

Rate limit / non-retried HTTP error
  ------------------------------> raise JDParsingError
```

Success is not the same as valid JD

A chain that returns status="needs_correction" has successfully classified the input. The utility sets succeeded=True because no exception occurred. The service returns that result immediately. Retrying unrelated text would waste requests.

Two containers, two responsibilities

Container	What it records
AttemptResult	One pipeline: returned value, attempts, succeeded.
JDParsingOutcome	Whole parsing workflow: JDParseResult, primary_attempts, fallback_attempts, exhausted.

exhausted=False does not mean the JD was valid. Blank input and normal correction responses also use False. It only means both retry paths did not run out of recoverable attempts.

Small class details

self._primary stores an injected chain; the underscore marks an internal attribute by convention. **@staticmethod** means _invoke does not need instance state. The assertions check the helper invariant that a successful parser attempt supplies a value; actual input validation comes from Pydantic, not assert.

Worked traces and error routing

Scenario	Primary	Fallback	Outcome
Valid JD on first attempt	1	0	valid; exhausted=False
Unrelated text classified normally	1	0	needs_correction; exhausted=False
Malformed output, then valid output	2	0	valid; exhausted=False
Primary fails 3 times; fallback works	3	1	valid; exhausted=False
Both paths repeatedly fail validation	3	2	needs_correction; exhausted=True
Blank or oversized input	0	0	needs_correction; exhausted=False
Primary receives HTTP 401 or 429	1 call	0	JDParsingError raised

Which failures get another attempt?

The current RETRYABLE_ERRORS tuple includes APIConnectionError (including SDK timeout subclasses), InternalServerError, OutputParserException, and Pydantic ValidationError. After primary exhaustion, fallback uses the same retry policy.

RateLimitError is handled outside the helper: the service raises a safe message immediately. Other APIStatusError exceptions, such as authentication or most HTTP 400 errors, also become JDParsingError. Exhausted 5xx errors are caught inside the helper first because InternalServerError is retryable.

Why catch order matters

```
except RateLimitError:
    raise JDParsingError("...rate-limited...") from None
except APIStatusError:
    raise JDParsingError("...provider rejected...") from None
```

RateLimitError is a kind of APIStatusError, so its specific handler must come first. **from None** suppresses the chained exception context in normal traceback display. The UI displays our safe message, not the raw provider payload.

An important limit of this implementation

There is no backoff yet, and unhandled programming errors still propagate. JDParsingError does not carry attempt counts, so the service cannot report detailed counts on that exception path. Five calls is a maximum for the recoverable two-pipeline path, not a promise that every failure gets five calls.

Understand the tests

Start with **tests/unit/test_reliability.py**, then read **tests/unit/test_jd_parser_service.py**. These tests use fake operations so failure behavior is repeatable and free of provider charges.

A fake runnable replaces Groq

```
pending = iter(outputs)
calls = []

def invoke(inputs):
    calls.append(inputs)
    return next(pending)

return RunnableLambda(invoke), calls
```

The service requires an object with invoke(); RunnableLambda supplies that interface around a normal Python function. Each call consumes the next queued output. The calls list independently records actual invocations.

Trace this test setup

```
primary, primary_calls = fake_chain([{}, {}, {}])
fallback, fallback_calls = fake_chain([valid_result()])
outcome = JDParsingService(primary, fallback).parse("JD")
```

Each empty dictionary fails JDParseResult.model_validate(), raising ValidationError. The utility catches that retryable error three times. Then the service runs the fallback, which returns a valid result once. Expected counts: primary=3, fallback=1.

Why check calls as well as counters?

Reported counts could be wrong. Asserting len(primary_calls) == 3 checks actual behavior, not only the numbers returned by the implementation. Tests also check that correction responses skip fallback, rejected requests stop early, and blank input makes zero calls.

What these tests cannot prove

They do not establish that an LLM extracts the right skills or resists every injected instruction. The default-to-mid rule lives in the prompt, not in the retry helper. Real-provider smoke checks and candidate review address different risks from these unit tests.

```
uv run pytest tests/unit/test_reliability.py -q
uv run pytest tests/unit/test_jd_parser_service.py -q
uv run pytest -q
```

Revision guide and next steps

Read the code in this order

1. Read AttemptResult: what can the helper return?
2. Read the attempt loop: when does it stop?
3. Read JDParsingOutcome: what does the workflow return?
4. Read _invoke(): what does one attempt actually do?
5. Read parse(): follow primary, fallback, then terminal behavior.
6. Read the tests: substitute queued outputs for the real model.

Check your understanding

Q1. Why pass a lambda rather than the result of _invoke()?
Q2. Does a correction response trigger fallback?
Q3. If the primary succeeds on attempt 2, how many fallback calls run?
Q4. Does exhausted=False prove the input is a valid JD?
Q5. Why does the service accept injected chains?

Answers

A1. The utility needs a callable it can execute repeatedly.
A2. No. A validated correction response returns normally.
A3. Zero. The service returns the primary result.
A4. No. Check result.status separately.
A5. It separates construction from policy and allows fake chains in tests.

Keep the learning model small

Think of the implementation as three ordinary operations: **make one attempt**, **repeat within a limit**, and **choose primary or fallback**. The types and wrappers describe those operations; they are not additional AI systems.

Current behavior to remember

SDK retries are disabled in providers/groq.py. Default limits are three primary and two fallback attempts. Changing JSON-schema mode to prompt-plus-parser fallback changes the output mechanism, not the underlying model. Confirmed requirements stay draft until a validated plan exists.